Building Scalable Real-World Robot Data Generation via Compositional Simulation

Yiran Qin[1,2,6*], **Jiahua Ma**[1*], **Li Kang**[3*], **Wenzhan Li**[1*]
Yihang Jiao[1], **Xin Wen**[1], **Xiufeng Song**[3], **Heng Zhou**[4], **Zhemeng Zhang**[5]
Zhenfei Yin[6], **Xihui Liu**[5], **Philip Torr**[6], **Ruimao Zhang**[1†]
[1]Sun Yat-sen University [2]CUHK-Shenzhen
[3]Shanghai Jiao Tong University [4]USTC
[5]The University of Hong Kong [6]University of Oxford
[*]Equal contribution [†]Corresponding author

Abstract: Recent advancements in foundational models, such as large language models and world models, have greatly enhanced the capabilities of robotics, enabling robots to autonomously perform complex tasks. However, acquiring large-scale, high-quality training data for robotics remains a challenge, as it often requires substantial manual effort and is limited in its coverage of diverse real-world environments. To address this, we propose a novel hybrid approach called ***Compositional Simulation***, which combines classical simulation and neural simulation to generate action-aligned video data with real-world visual appearance. Our approach utilizes a real-sim-real data augmentation pipeline, leveraging a small amount of real-world data to generate larger and more diverse training datasets that cover a broader range of object and spatial configurations. We train a neural simulator to transform classical simulation videos into real-world appearance representations while preserving the underlying action trajectories. Through experiments on real-robot tabletop manipulation under a limited real-data budget, we show that our method reduces the appearance-level sim-to-real gap and yields higher success rates for real-world policy training, including improved transfer to spatial layouts and objects absent from real demonstrations but covered by synthetic training data. Our approach offers a practical path toward augmenting real-world robot training data and narrowing the gap between simulated and real-world robotics. Project website: https://faceong.github.io/ComSim/.

Keywords: Sim-to-Real, Robot Data Generation, World Models, Robotic Manipulation

1 Introduction

Recent advances in foundation models [1, 2, 3, 4, 5, 6] have driven progress in robotics [7, 8], yet large-scale, action-aligned video data remain a bottleneck: collecting such data via real-world teleoperation is expensive and insufficient to cover the diversity of real environments. A common alternative is to scale data through classical simulators [9, 10, 11, 12, 13], which provide precise action–video pairs at scale. However, even with carefully designed sim-and-real co-training recipes [14], the appearance and physics gap means that naively mixing simulated and real data does not reliably help and can degrade real-world performance; the achievable benefit depends strongly on the visual fidelity of the simulated data.

To bridge this gap, neural simulators [5, 15, 6] based on video generation models [16, 17, 18] have recently been proposed as a solution. These simulators generate corresponding video data from input trajectories or action signals, producing action-video pairs for training. Although the generated

10th Conference on Robot Learning (CoRL 2026), Austin, Texas, USA.

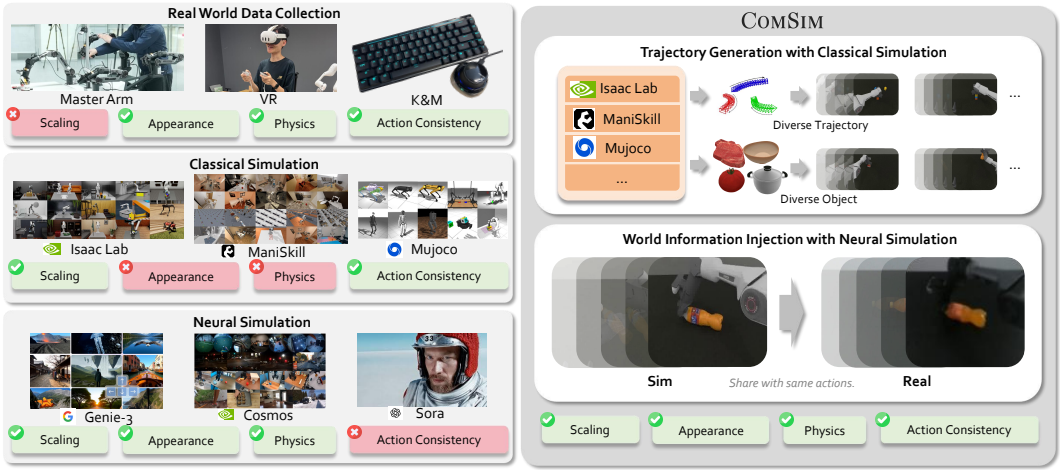


Figure 1: There are three main sources of real-world robotic data: (1) direct human collection, which yields high-quality samples but is labor-intensive; (2) classical simulators, which generate large datasets but suffer from appearance and physics gaps to reality; and (3) neural simulators trained on real data, which reduce appearance differences but struggle with action-conditioned video generation, leading to weak action–video consistency. We introduce the concept of *Compositional Simulation*, a hybrid approach that narrows the appearance gap between classical simulation and the real world via compositional simulation, while inheriting the action precision of the classical simulator.

videos appear visually consistent with the real world, issues like hallucination—where videos lack physical consistency and lead to poor action control—undermine the quality of the generated data.

In this work, we introduce the concept of ***Compositional Simulation*** shown in Fig. 1, advocating for a hybrid approach that combines the strengths of classical simulation and neural simulation. This approach aims to generate action-aligned video pairs while reducing the visual gap to real observations; action labels and simulated dynamics are retained, while video alignment is evaluated empirically. We propose a real-sim-real data augmentation pipeline that utilizes a small amount of real-world data to create training datasets for policy models. These datasets are designed to cover a broader distribution of real-world scenarios.

The pipeline has two stages. First, a small set of real-world trajectories is replayed in a digital twin of the real platform to produce paired $(\mathcal{V}_{\text{sim}}, \mathcal{V}_{\text{real}}, \mathcal{A})$ tuples, on which we train a neural simulator with Compositional Dynamic Video Generation. Second, large-scale action–video pairs are generated by action-primitive scheduling in the classical simulator and re-rendered into real-world appearance by the trained neural simulator. Our main contributions are:

- *Compositional Formulation.* We introduce the concept of *Compositional Simulation*, a hybrid approach that narrows the appearance-level gap between classical simulation and the real world via compositional simulation, while preserving the action trajectories produced by the classical simulator.

- *Data Pipeline & Model.* We propose a real–sim–real data augmentation pipeline that builds a neural simulator with Compositional Dynamic Video Generation, which is designed to keep the generated video aligned with the conditioning action trajectory while reducing the appearance-level sim-to-real gap.

- *Experimental Results.* Experiments on real-robot tabletop manipulation under a limited real-data budget show that *Compositional Simulation* improves policy success rates and transfer to spatial layouts and object variations covered by synthetic data but absent from real demonstrations relative to real-only and raw-simulation baselines.

2 Related Work

Data Augmentation for Robotic Manipulation. Real-to-sim-to-real pipelines—URDFormer [19], RialTo [20], ACDC [21]—construct simulation environments from real-world data, while MimicGen [22] and DemoGen [23] adapt action trajectories to novel object configurations but render observations either in simulation or by replaying real frames, leaving the appearance gap open. A complementary line co-trains policies on simulated and real data [14], which is most effective when the simulated observations are already visually close to the real ones. Unsupervised domain-adaptation methods such as CycleGAN [24] translate appearance without paired data, but provide no guarantee that control-relevant content is preserved. Closest to our setting are re-rendering pipelines such as ReBot [25], which similarly replay a small real dataset in simulation and re-render it into real-world style; *Compositional Simulation* differs in using a Stable Diffusion 1.5 U-Net with compositionally combined control- and visual-dynamics scores and in explicitly studying spatial and object generalization (Tab. 6, Appendix 12).

World Simulators for Robotic Manipulation. Generative video models [6, 26] offer a cost-effective way to synthesize policy training data: UniPi [7] and AVDC [27] cast planning as text-to-video generation; UniSim [8] learns a unified text- and control-conditioned simulator; RoboDreamer [28] targets compositional generalization via text parsing; and IRASim [29] performs trajectory-conditioned video generation focused on arm motion. Specialized action-conditioned world models—Ctrl-World [30], EnerVerse-AC [31], Genie-Envisioner [32], DreamDojo [33]—roll out future observations directly from actions, but must learn physically consistent dynamics from data and can hallucinate under distribution shift. In contrast, *Compositional Simulation* does not predict dynamics: it keeps action and contact dynamics fixed to those of the classical simulator and uses the neural model only to translate the simulated rollout into real-world appearance. A broader discussion of robotic simulation platforms and large-scale policy architectures is provided in Appendix 13.

3 Compositional World Simulation

3.1 Problem Formulation

Collecting real-world manipulation data is expensive, so a common workaround is to scale data via classical simulators [12, 13, 34, 10, 11], optionally driven by LLM-generated primitives, and to pre-train or co-train policies on the resulting trajectories. Let $\mathcal{D}_{\text{sim}} = \{(v_i, a_i)\}_{i=1}^{N}$ and $\mathcal{D}_{\text{real}} = \{(v'_j, a'_j)\}_{j=1}^{M}$ denote the simulated and real datasets, where each v_i is a video and a_i its action sequence. Naively mixing $\mathcal{D}_{\text{sim}}$ with $\mathcal{D}_{\text{real}}$ can fail to improve transfer because of a two-faceted domain gap: an *appearance gap* (textures, lighting, materials, sensor characteristics) and a *physics gap* (contact, friction, compliance, object dynamics). Action-conditioned video generation models address appearance directly but suffer from hallucinations, 3D inconsistency, and weak action control, producing video-action pairs that are not reliably aligned. We therefore propose a *compositional simulation* approach: keep the classical simulator's physics, and learn a neural simulator $\mathcal{N}$ that re-renders its rollouts into the real-world appearance distribution while preserving the original action labels unchanged,

$$\mathcal{N}(\mathcal{D}_{\text{sim}}) \approx \mathcal{D}_{\text{real}}, \tag{1}$$

so that the appearance gap is reduced while the action and contact dynamics are inherited from the simulator. Closing the physics gap is outside the scope of this work.

3.2 Real2Sim Data Collection

Training $\mathcal{N}$ requires paired tuples $(\mathcal{V}_{\text{sim}}, \mathcal{V}_{\text{real}}, \mathcal{A})$, where the simulator and the real platform execute the same action sequence $\mathcal{A}$. To obtain such pairs we build a digital twin of the real-world setup (Fig. 2, Left) with strict alignment at three levels: (i) *background and object alignment* (desktop/wall colors and asset geometry are matched to real-world scale); (ii) *camera calibration and alignment* (intrinsics and extrinsics matched to the real RGB-D camera); and (iii) *object position alignment* (per-

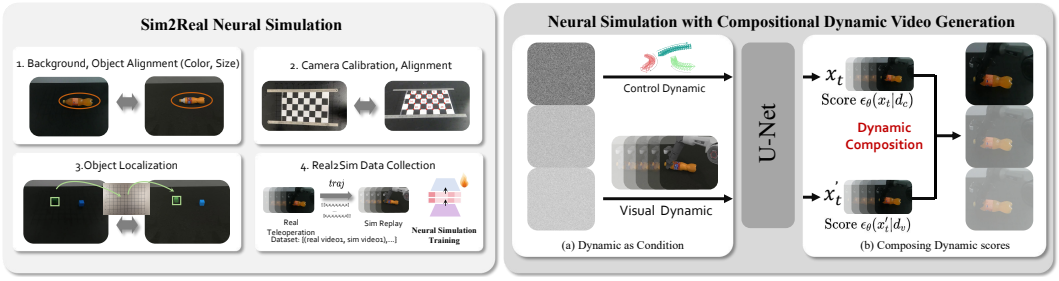


Figure 2: (Left) Alignment between real-world and simulation: trajectories collected in the real world are replayed in simulation to generate paired video data for training the sim-to-real neural simulator. (Right) A Stable Diffusion 1.5 U-Net estimates unconditional, visual-conditioned, and action-conditioned scores, which are composed during sampling. FRESCO provides post-denoising temporal correspondence.

trial object localizations from the real scene are transferred into the simulator at task initialization). Full procedural details are given in Appendix 11.

3.3 Sim2Real Compositional Dynamic Video Generation

To encourage action alignment during appearance transfer, we train the neural simulator using Compositional Dynamic Video Generation. The neural simulator is a conditional latent diffusion model: we fine-tune a pretrained Stable Diffusion 1.5 U-Net (optimizer and schedule in Appendix 9) to denoise the latent of the target real-world video. We expose two conditioning signals, illustrated in Fig. 2 (Right): **Control Dynamics** c_c, the action sequence $\mathcal{A}$ executed by the robot, and **Visual Dynamics** c_v, the corresponding classical-simulation observation stream $\mathcal{V}_{\text{sim}}$. At sampling time, we use a compositional, classifier-free-guidance-style formulation [35]: the two conditions are evaluated separately, and their noise predictions are combined. Concretely, let $\epsilon_\theta(z_t, t, \cdot)$ be the predicted noise for the noisy latent z_t at timestep t. We form a *composed noise prediction*

$$\hat{\epsilon}_\theta(z_t, t) = \epsilon_\theta(z_t, t, \varnothing) + w_v \left(\epsilon_\theta(z_t, t, c_v) - \epsilon_\theta(z_t, t, \varnothing)\right) + w_c \left(\epsilon_\theta(z_t, t, c_c) - \epsilon_\theta(z_t, t, \varnothing)\right), \quad (2)$$

where $\varnothing$ denotes the absence of either condition and the guidance weights $w_v, w_c \geq 0$ trade off visual realism against action fidelity. This decomposition is what we refer to as *Dynamic Guidance*: setting $w_c = 0$ recovers the visual-dynamics-only variant (Ours-VD), $w_v = 0$ recovers the control-dynamics-only variant (Ours-CD), and using both recovers the full model (Ours-Full), exactly matching the ablation in Sec. 4. Composing the two conditional scores at sampling time, rather than relying on a single fused condition, lets us tune the balance between appearance transfer and action consistency without retraining.

Specifically, we collect data from the real-world simulation platform in Sec. 3.2 to obtain the pair $(\mathcal{V}_{\text{real}}, \mathcal{A})$. These action data are then replayed in the corresponding classical simulation environment to generate the tuple $(\mathcal{V}_{\text{sim}}, \mathcal{V}_{\text{real}}, \mathcal{A})$. The paired corpus contains 200 trajectory pairs in total across 10 source tasks and supports one shared neural simulator; the downstream synthetic-data budget is separately 200 demonstrations per task. To optimize the neural simulator for Sim2Real data generation, we aim to minimize the discrepancy between the simulated and real-world videos, while maintaining the correct action alignment. Since the actions in both $\mathcal{V}_{\text{sim}}$ and $\mathcal{V}_{\text{real}}$ are already aligned, we focus solely on optimizing the video consistency. The optimization objective is formulated as:

$$\mathcal{L}_{\text{sim2real}} = \mathcal{L}_{\text{video}}(f_{\mathcal{N}}(\mathcal{V}_{\text{sim}}, \mathcal{A}, \theta), \mathcal{V}_{\text{real}}) \quad (3)$$

where $\mathcal{L}_{\text{video}}$ measures the difference between the generated and real-world video, and θ represents the neural simulator's parameters. In practice $\mathcal{L}_{\text{video}}$ is a weighted sum of the diffusion denoising loss together with perceptual and pixel-wise reconstruction terms; the exact weights are reported in

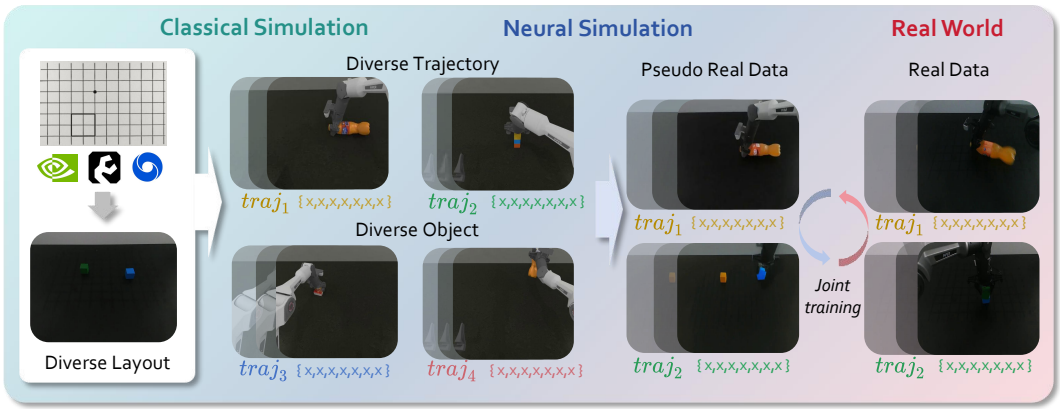


Figure 3: **Real World Deployment with *Compositional Simulation*.** Large volumes of $(\mathcal{V}_{\text{sim}}, \mathcal{A})$ pairs are collected from the classical simulator and transformed into corresponding $(\mathcal{V}_{\text{real}}, \mathcal{A})$ pairs, referred to as *Pseudo Real Data*. These data, together with a small amount of real-world data, are used to train policies with improved success rates and generalization.

Appendix 9. The conditions c_c and c_v are dropped independently with probability 0.1 during training so that the unconditional and single-condition scores in Eq. 2 are all available at sampling time. The simulation video enters through cross-attention, while a three-layer MLP encodes actions and injects them through ControlNet [36] residual blocks. Inference requires only $\mathcal{V}_{\text{sim}}$ and $\mathcal{A}$, with no task label; $(w_v, w_c) = (1.5, 1.0)$. At inference we denoise with the composed score of Eq. 2 and then apply the FRESCO [37] post-processing step to improve temporal coherence across frames. By minimizing this loss, the neural simulator learns to map simulated rollouts to the real-world appearance distribution while preserving the action trajectory inherited from the classical simulator; this objective does not, by itself, certify that fine-grained contact or force dynamics match the real world.

3.4 Data Generation with Rule-Based Simulation

To further scale up the data collection pipeline, we employ RoboTwin [38], a SAPIEN-based [39] dual-arm manipulation simulation environment. It provides a rich library of digital assets and supports diverse trajectory distributions, making it well-suited for synthesizing large-scale visuomotor datasets. By systematically varying environmental conditions, object initialization states, and agent actions, we generate an extensive set of trajectories and corresponding videos that cover a broad spectrum of real-world task scenarios.

Specifically, we define a comprehensive set of interaction rules, referred to as action primitives, governing how agents and objects interact within the simulation. These primitives serve as the atomic building blocks of complex behaviors, capturing low-level manipulations (e.g., grasp, push, align) as well as higher-order skills (e.g., stack blocks). We curate a suite of RoboTwin tasks and adapt them to support richer interaction patterns and object configurations, enabling a broader spectrum of physical reasoning scenarios. To automate the generation of complex behaviors, we employ GPT-5 [1] to synthesize executable code composed of these action primitives, while integrating compositional constraints [11] to ensure semantic correctness and physical feasibility. The action primitives encompass a variety of object types and interaction modalities, enabling diverse scenario generation. For each task, we construct a rich collection of trajectories τ_s spanning the action space, and carefully tune the primitive-based generation process to achieve comprehensive coverage. This samples varied behaviors in the simulation, including different object initializations and heterogeneous object categories.

The resulting dataset comprises temporally synchronized camera observations, corresponding action and state sequences. These elements are strictly aligned at the behavioral level, ensuring that every visual frame is paired with its underlying control command. Although the trajectories and interactions

in simulation are faithful to their intended semantics, the rendered appearance of the videos still differs from real-world imagery due to discrepancies in lighting, textures, and sensor noise. To bridge this domain gap, we pass the simulated observation streams v_s through a neural simulator $\mathcal{N}$, which refines their visual characteristics while retaining the action labels; video–action consistency is evaluated in Sec. 4.

Real-World Deployment. As illustrated in Fig. 3, the trained neural simulator transforms a large pool of $(\mathcal{V}_{\text{sim}}, \mathcal{A})$ pairs from the classical simulator into corresponding $(\mathcal{V}_{\text{real}}, \mathcal{A})$ *Pseudo Real Data*, which are jointly used with a small amount of real demonstrations to train the downstream policy; the resulting success rates and generalization behavior are reported in Sec. 4.2, and pseudo-code for the full pipeline is given in Appendix 16.

4 Experiments

4.1 Sim2Real Transfer via Neural Simulation

Baselines and evaluation scope. We compare SD1.5 video-to-video translation with FRESCO (Baseline), control-only conditioning (Ours-CD), visual-only conditioning (Ours-VD), and their composition (Ours-Full). We evaluate appearance and action consistency through qualitative comparisons and action recovery, and assess downstream utility through real-robot policy evaluation. The original frame-split visual metrics are retained in Appendix 19. Because adjacent frames can cross that split, those metrics are diagnostic and do not establish held-out-trajectory generalization.

Qualitative Results. The original Baseline is an SD1.5 video-to-video model with FRESCO. Fig. 4 compares the methods on four representative tasks covering object-attribute and background variations. Conventional diffusion baselines preserve scene layout but introduce severe appearance hallucinations. Ours-CD reproduces the target manipulation actions but, lacking visual conditioning, drifts in viewpoint, object identity, and background. Ours-VD recovers the realistic appearance yet exhibits action-timing artifacts and physically implausible motion blur. Ours-Full improves both appearance and action alignment in these examples.

Action Consistency. Beyond visual realism, we directly verify that the generated videos preserve control-relevant motion by recovering an action signal (end-effector trajectory via gripper-keypoint detection and open/close timing) and comparing it against the conditioning sequence $\mathcal{A}$. Tab. 1 reports

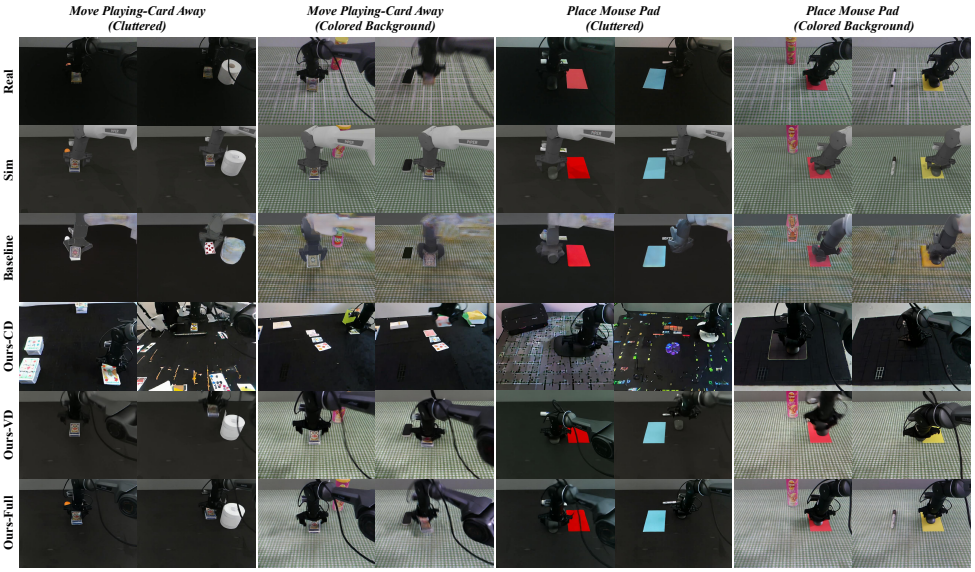


Figure 4: Visual comparison of generated results across four representative tasks. Rows: Real, Sim, Baseline, Ours-CD, Ours-VD, Ours-Full.

Table 1: Action-consistency of the generated videos vs. the conditioning action sequence. Lower trajectory and timing error indicate better preservation of control-relevant motion; higher action-recovery accuracy indicates a more faithfully replayable trajectory.

Method	EE-Traj. Error (cm) ↓	Gripper-Timing Err. (frames) ↓	Action-Recovery Acc. (%) ↑
Baseline	3.82	9.4	54.7
Ours-CD	1.12	2.3	79.8
Ours-VD	2.61	6.5	68.2
Ours-Full	**0.69**	**1.2**	**94.3**

Table 2: Quantitative Results of Our Real-world Experiments. Each cell reports single-run successes over 30 trials. "OOD" abbreviates out-of-distribution; columns correspond to six data-mixture regimes. Best result per row in **bold**.

Task	Spatial	10 Real	20 Real	200 Sim Pretrain + 10 Real	10 Real + 200 Sim	10 Real + 200 Pseudo Real	200 Pseudo Real (Zero Shot)
Shake Bottle	In Domain	9/30	17/30	12/30	6/30	**28/30**	10/30
	OOD	0/30	1/30	0/30	0/30	**12/30**	5/30
Stack Blocks Two	In Domain	5/30	13/30	8/30	2/30	**15/30**	7/30
	OOD	0/30	0/30	0/30	0/30	**6/30**	3/30
Move Playing-Card Away	In Domain	12/30	24/30	15/30	6/30	**29/30**	18/30
	OOD	0/30	3/30	2/30	1/30	**17/30**	9/30
Move Playing-Card Away (Cluttered)	In Domain	7/30	18/30	11/30	3/30	**25/30**	15/30
	OOD	0/30	1/30	1/30	1/30	**16/30**	8/30
Move Playing-Card Away (Colored Background)	In Domain	10/30	20/30	6/30	8/30	**23/30**	17/30
	OOD	0/30	2/30	1/30	2/30	**17/30**	10/30
Place Mouse Pad (Cluttered)	In Domain	4/30	15/30	6/30	7/30	**19/30**	8/30
	OOD	0/30	0/30	1/30	1/30	**10/30**	5/30
Place Mouse Pad (Colored Background)	In Domain	7/30	18/30	8/30	5/30	**22/30**	12/30
	OOD	0/30	1/30	1/30	1/30	**14/30**	9/30
Handover Bottle (Cluttered)	In Domain	3/30	8/30	1/30	1/30	**13/30**	11/30
	OOD	0/30	0/30	0/30	0/30	**5/30**	3/30

the four-method action-recovery diagnostic. The Baseline drifts on every metric due to the absence of control conditioning. Ours-CD improves open/close timing but its degraded appearance impairs keypoint detection and inflates the recovered trajectory error. Ours-VD produces realistic frames so detection is reliable, but the recovered motion drifts from $\mathcal{A}$ without control conditioning. Only Ours-Full attains the lowest EE-trajectory error (0.69 cm), the lowest gripper-timing error (1.2 frames), and the highest action-recovery accuracy (94.3%) in that original diagnostic—a substantial improvement over the Baseline (3.82 cm, 9.4 frames, 54.7%).

4.2 Real-World Execution with Compositional Simulation

Baselines. Under a tight real-world demonstration budget, we train a separate single-task Diffusion Policy (DP) [40] for each task and data-mixture regime: (1) *10 Real*, (2) *20 Real*, (3) *200 Sim Pretrain + 10 Real*, (4) *10 Real + 200 Sim* (joint training from scratch), (5) *10 Real + 200 Pseudo Real*, and (6) *200 Pseudo Real* (no real policy demonstrations). All Sim demonstrations are from RoboTwin [38]; Pseudo Real demonstrations come from our pipeline.

Quantitative Results. Tab. 2 reports per-task DP success rates over a fixed set of 30 real-world trials, separated into in-domain and OOD spatial conditions. Several consistent patterns emerge. (i) Real-only training scales monotonically with budget (*10 Real* → *20 Real*) but is bounded by the absolute amount of real data, and never closes the gap to Pseudo-Real-augmented training. (ii) Plain RoboTwin simulation—either as pre-training or as joint training—brings only marginal gains and on several tasks degrades performance relative to *20 Real*, consistent with a remaining sim-to-real gap. (iii) Substituting the simulated split with our Pseudo-Real demonstrations (*10 Real + 200 Pseudo Real*) yields the largest in-domain gains on every task, improving over *20 Real* while using half as many real policy demonstrations. (iv) Even the zero-shot *200 Pseudo Real* regime, which uses no real policy demonstrations but retains real supervision for the neural simulator, delivers non-trivial

Table 3: DP transfer to objects absent from real demonstrations but covered by synthetic training data, across six data mixtures. Each cell reports single-run successes over 30 trials.

Task	**10 Real**	**20 Real**	**200 Sim Pretrain + 10 Real**	**10 Real + 200 Sim**	**10 Real + 200 Pseudo Real**	**200 Pseudo Real (Zero Shot)**
Shake Bottle	0/30	0/30	0/30	0/30	**15/30**	9/30
Move Playing-Card Away	1/30	2/30	1/30	0/30	**21/30**	11/30

Table 4: Three-seed downstream ablation: mean success (%). Spatial ID/OOD is relative to real demonstrations.

Task	Ours-VD (ID/OOD)	Ours-Full (ID/OOD)	Gain (ID/OOD)
Shake Bottle	63.3/46.7	83.3/70.0	+20.0/+23.3
Move Playing-Card Away	60.0/43.3	76.7/63.3	+16.7/+20.0

success rates, suggesting that the Pseudo-Real distribution alone is a usable training signal when additional real policy demonstrations are unavailable. These original results are single-run counts; the additional three-seed experiments below assess the VD/Full comparison separately.

4.3 Transfer of Synthetic Coverage to Real Execution

Spatial transfer. Synthetic policy-training data cover the OOD region; OOD is defined relative to real demonstrations, not the full training pipeline. The OOD column of Tab. 2 probes object initial positions *outside* the spatial region covered by the real demonstrations—a setting outside the real-demonstration coverage. Real-only training (*10 Real*, *20 Real*) and the two Sim-augmented conditions stay at or near zero across all 8 tasks, indicating that neither doubling the real budget nor adding RoboTwin demonstrations restores spatial generalization once the test layouts leave the demonstrated region. In contrast, *10 Real + 200 Pseudo Real* lifts OOD success above the real-only and raw-simulation conditions on every task, with gains such as $0/30 \rightarrow 12/30$ on *Shake Bottle* and $0/30 \rightarrow 17/30$ on *Move Playing-Card Away*, supporting transfer of the synthetic spatial coverage to real execution.

Object transfer. Test objects are absent from both neural-simulator real training pairs and real DP demonstrations, but present in RoboTwin assets and Sim/Pseudo-Real policy-training data. At real-world evaluation, the Fanta bottle is replaced with Coca-Cola, Sprite, and Nongfu Spring; the blue playing card with a red one. Tab. 3 shows that adding Pseudo-Real demonstrations lifts object-transfer success from $\leq 2/30$ to $\mathbf{15}$–$\mathbf{21}/\mathbf{30}$, while RoboTwin simulation brings no improvement. Rollout visualizations are in Appendix 17; per-scene cost is reported in Sec. 4.5, and conceptual positioning in Appendix 12.

4.4 Downstream Ablation

Control-branch contribution. We compare Ours-VD and Ours-Full using 10 real and 200 Pseudo-Real demonstrations per task. Each result in Table 4 averages three seeds with 30 real-robot trials per seed. Full improves on VD by 16.7–23.3 percentage points across the two tasks and spatial splits, supporting the importance of action conditioning beyond appearance quality. These additional runs are distinct from the original single-run counts in Table 2.

4.5 Implementation and Computational Cost

Policy training. We use Diffusion Policy (DP) [40], a generative method based on imitation learning. We employ a CNN-based Diffusion Policy as the backbone of our visuomotor model. The prediction horizon is set to 8, with 3 observation steps and 6 action steps. For data loading, we use a batch size of 256. The optimizer is `torch.optim.AdamW` with a learning rate of 1.0×10^{-4},

$(\beta_1, \beta_2) = (0.95, 0.999)$, and $\epsilon = 1.0 \times 10^{-8}$. A learning-rate warmup is applied for the first 500 steps, followed by 300 training epochs for all benchmark tasks.

Each policy is trained independently on a single NVIDIA H200 GPU for 300 epochs. As a reference, using a dataset of roughly 200 demonstration episodes (average length $\approx$ 300), training a single policy for 300 epochs takes about 20 hours.

Generation and setup cost. Because *Compositional Simulation* is positioned as a route to *scalable* data generation, we make its practical cost explicit in Tab. 5. Training the neural simulator takes roughly 10 hours on a single NVIDIA H200 GPU, and training each Diffusion Policy takes roughly 20 hours on the same hardware (Appendix 9). Two further quantities ultimately govern how the pipeline scales to new scenes: (i) the wall-clock cost of generating one pseudo-real episode through the neural simulator, and (ii) the human effort needed to build an aligned digital twin for a new scene (Appendix 11). We measure both: pseudo-real generation takes ≈ 45 s per episode on a single H200, so generating 200 episodes takes about 2.5 GPU-hours before regeneration or rejection overhead; and the rule-based digital-twin alignment pipeline of Appendix 11 requires ≈ 1.5 h of human effort per new scene. These setup and generation costs must be accounted for when assessing scalability.

Table 5: **Computational and human cost of the pipeline.** Compute figures are measured on a single NVIDIA H200 GPU (Appendix 9); the digital-twin alignment cost reflects the rule-based pipeline of Appendix 11.

Stage	Resource	Cost
Neural-simulator training	1×H200	$\approx$ 10 h
Diffusion Policy training (per policy)	1×H200	$\approx$ 20 h
Pseudo-real generation (per episode)	1×H200	$\approx$ 45 s
Digital-twin alignment (per new scene)	human effort	$\approx$ 1.5 h

5 Limitations

Our pipeline targets the appearance gap only and inherits the classical simulator's physics, so tasks requiring fine-grained contact or compliance feedback fall outside its direct regime. Each new scene also requires rebuilding an aligned digital twin, and our evaluation covers eight task/scene settings on a single dual-arm platform with Diffusion Policy. Benefits for pretrained vision–language–action (VLA) policies remain untested; their visual pretraining may reduce the appearance gap. Synthetic coverage should not be interpreted as extrapolation beyond the pipeline training distribution. An extended discussion and future-work plan are in Appendix 14.

6 Conclusion

We presented *Compositional Simulation*, a hybrid real–sim–real framework that re-renders classical-simulator rollouts into action-aligned, real-appearance videos via separate visual- and control-dynamics branches. On real-robot tabletop manipulation with 10 real policy demonstrations per task and a separately trained neural simulator, training Diffusion Policy with the resulting Pseudo-Real data outperforms real-only and Sim-augmented baselines, lifts OOD spatial success from near-zero to double digits across all eight task/scene settings, and improves transfer to objects absent from real demonstrations but covered by synthetic training. A detailed future-work plan is in Appendix 14.

Ethics & Reproducibility. Experiments involve only standard lab manipulation of everyday objects; no human subjects or sensitive data are involved. Policy training and compute costs are reported in Sec. 4.5; neural-simulator training and the real-world protocol are detailed in the appendix.

References

[1] OpenAI. Gpt-5 system card (updated august 13, 2025). https://cdn.openai.com/gpt-5-system-card.pdf, Aug 2025.

[2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[3] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[4] OpenAI. Creating video from text. https://openai.com/index/sora/, 2024.

[5] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *ICML*, 2024.

[6] N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.

[7] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.

[8] M. Yang, Y. Du, K. Ghasemipour, J. Tompson, D. Schuurmans, and P. Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2023.

[9] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[10] Y. Mu, T. Chen, S. Peng, Z. Chen, Z. Gao, Y. Zou, L. Lin, Z. Xie, and P. Luo. Robotwin: Dual-arm robot benchmark with generative digital twins (early version). *arXiv preprint arXiv:2409.02920*, 2024.

[11] Y. Qin, L. Kang, X. Song, Z. Yin, X. Liu, X. Liu, R. Zhang, and L. Bai. Robofactory: Exploring embodied agent collaboration with compositional constraints. *arXiv preprint arXiv:2503.16408*, 2025.

[12] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi:10.1109/IROS.2012.6386109.

[13] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac gym: High performance GPU based physics simulation for robot learning. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/28dd2c7955ce926456240b2ff0100bde-Abstract-round2.html.

[14] A. Maddukuri, Z. Jiang, L. Y. Chen, S. Nasiriany, Y. Xie, Y. Fang, W. Huang, Z. Wang, Z. Xu, N. Chernyadev, S. Reed, K. Goldberg, A. Mandlekar, L. Fan, and Y. Zhu. Sim-and-real co-training: A simple recipe for vision-based robotic manipulation. In *Proceedings of Robotics: Science and Systems (RSS)*, Los Angeles, CA, USA, 2025.

[15] J. Yu, Y. Qin, X. Wang, P. Wan, D. Zhang, and X. Liu. Gamefactory: Creating new games with generative interactive videos, 2025.

[16] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.

[17] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[18] Z. Zheng, X. Peng, T. Yang, C. Shen, S. Li, H. Liu, Y. Zhou, T. Li, and Y. You. Open-sora: Democratizing efficient video production for all, March 2024. URL https://github.com/hpcaitech/Open-Sora.

[19] Z. Chen, A. Walsman, M. Memmel, K. Mo, A. Fang, K. Vemuri, A. Wu, D. Fox, and A. Gupta. Urdformer: A pipeline for constructing articulated simulation environments from real-world images. *arXiv preprint arXiv:2405.11656*, 2024.

[20] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. *arXiv preprint arXiv:2403.03949*, 2024.

[21] T. Dai, J. Wong, Y. Jiang, C. Wang, C. Gokmen, R. Zhang, J. Wu, and L. Fei-Fei. Automated creation of digital cousins for robust policy learning. *arXiv preprint arXiv:2410.07408*, 2024.

[22] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.

[23] Z. Xue, S. Deng, Z. Chen, Y. Wang, Z. Yuan, and H. Xu. Demogen: Synthetic demonstration generation for data-efficient visuomotor policy learning. *arXiv preprint arXiv:2502.16932*, 2025.

[24] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2223–2232, 2017.

[25] Y. Fang, Y. Yang, X. Zhu, K. Zheng, G. Bertasius, D. Szafir, and M. Ding. Rebot: Scaling robot learning with real-to-sim-to-real robotic video synthesis. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 11351–11358. IEEE, 2025.

[26] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. *arXiv preprint arXiv:2312.13139*, 2023.

[27] P.-C. Ko, J. Mao, Y. Du, S.-H. Sun, and J. B. Tenenbaum. Learning to act from actionless videos through dense correspondences. *arXiv preprint arXiv:2310.08576*, 2023.

[28] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan. Robodreamer: Learning compositional world models for robot imagination. *arXiv preprint arXiv:2404.12377*, 2024.

[29] F. Zhu, H. Wu, S. Guo, Y. Liu, C. Cheang, and T. Kong. Irasim: Learning interactive real-robot action simulators. *arXiv preprint arXiv:2406.14540*, 2024.

[30] Y. Guo, L. X. Shi, J. Chen, and C. Finn. Ctrl-world: A controllable generative world model for robot manipulation. *arXiv preprint arXiv:2510.10125*, 2025.

[31] Y. Jiang, S. Chen, S. Huang, L. Chen, P. Zhou, Y. Liao, X. He, C. Liu, H. Li, M. Yao, et al. Enerverse-ac: Envisioning embodied environments with action condition. *arXiv preprint arXiv:2505.09723*, 2025.

[32] Y. Liao, P. Zhou, S. Huang, D. Yang, S. Chen, Y. Jiang, Y. Hu, J. Cai, S. Liu, J. Luo, et al. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025.

[33] S. Gao, W. Liang, K. Zheng, A. Malik, S. Ye, S. Yu, W.-C. Tseng, Y. Dong, K. Mo, C.-H. Lin, et al. Dreamdojo: A generalist robot world model from large-scale human videos. *arXiv preprint arXiv:2602.06949*, 2026.

[34] J. Gu, F. Xiang, X. Li, Z. Ling, X. Liu, T. Mu, Y. Tang, S. Tao, X. Wei, Y. Yao, et al. Maniskill2: A unified benchmark for generalizable manipulation skills. In *The Eleventh International Conference on Learning Representations*, 2023.

[35] J. Ho and T. Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

[36] L. Zhang, A. Rao, and M. Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3836–3847, 2023.

[37] S. Yang, Y. Zhou, Z. Liu, and C. C. Loy. Fresco: Spatial-temporal correspondence for zero-shot video translation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8703–8712, 2024.

[38] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Q. Liang, Z. Li, X. Lin, Y. Ge, Z. Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[39] F. Xiang, Y. Qin, K. Mo, Y. Xia, H. Zhu, F. Liu, M. Liu, H. Jiang, Y. Yuan, H. Wang, L. Yi, A. X. Chang, L. J. Guibas, and H. Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[40] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion Policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023. doi:10.15607/RSS.2023.XIX.026.

[41] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[42] A. Szot, A. Clegg, E. Undersander, E. Wijmans, Y. Zhao, J. Turner, N. Maestre, M. Mukadam, D. Chaplot, O. Maksymets, A. Gokaslan, V. Vondrus, S. Dharur, F. Meier, W. Galuba, A. Chang, Z. Kira, V. Koltun, J. Malik, M. Savva, and D. Batra. Habitat 2.0: Training home assistants to rearrange their habitat. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[43] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, D. Gordon, Y. Zhu, A. Gupta, and A. Farhadi. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv*, 2017.

[44] C. Li, R. Zhang, J. Wong, C. Gokmen, S. Srivastava, R. Martín-Martín, C. Wang, G. Levine, M. Lingelbach, J. Sun, M. Anvari, M. Hwang, M. Sharma, A. Aydin, D. Bansal, S. Hunter, K.-Y. Kim, A. Lou, C. R. Matthews, I. Villa-Renteria, J. H. Tang, C. Tang, F. Xia, S. Savarese, H. Gweon, K. Liu, J. Wu, and L. Fei-Fei. BEHAVIOR-1k: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation. In *6th Annual Conference on Robot Learning*, 2022. URL https://openreview.net/forum?id=_8DoIe8G3t.

[45] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki. 3d diffuser actor: Policy diffusion with 3d scene representations. *arXiv preprint arXiv:2402.10885*, 2024.

[46] Z. Liang, Y. Mu, M. Ding, F. Ni, M. Tomizuka, and P. Luo. Adaptdiffuser: Diffusion models as adaptive self-evolving planners. In *International Conference on Machine Learning*, pages 20725–20745. PMLR, 2023.

[47] Z. Liang, Y. Mu, H. Ma, M. Tomizuka, M. Ding, and P. Luo. Skilldiffuser: Interpretable hierarchical planning via skill abstractions in diffusion-based task execution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16467–16476, 2024.

[48] Z. Liang, Y. Mu, Y. Wang, T. Chen, W. Shao, W. Zhan, M. Tomizuka, P. Luo, and M. Ding. Dexhanddiff: Interaction-aware diffusion planning for adaptive dexterous manipulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1745–1755, 2025.

[49] C. Wang, H. Fang, H.-S. Fang, and C. Lu. Rise: 3d perception makes real-world robot imitation simple and effective. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2870–2877. IEEE, 2024.

[50] J. Wen, Y. Zhu, J. Li, Z. Tang, C. Shen, and F. Feng. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.

[51] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.

[52] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[53] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[54] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. Rdt-1b: a diffusion foundation model for bimanual manipulation. *arXiv preprint arXiv:2410.07864*, 2024.

[55] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[56] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, et al. Openvla: An open-source vision-language-action model. In *8th Annual Conference on Robot Learning*, 2024.

[57] Q. Li, Y. Liang, Z. Wang, L. Luo, X. Chen, M. Liao, F. Wei, Y. Deng, S. Xu, Y. Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.

[58] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.

[59] S. Ye, J. Jang, B. Jeon, S. J. Joo, J. Yang, B. Peng, A. Mandlekar, R. Tan, Y.-W. Chao, B. Y. Lin, et al. Latent action pretraining from videos. In *CoRL 2024 Workshop on Whole-body Control and Bimanual Manipulation: Applications in Humanoids and Beyond*, 2024.

[60] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[61] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018.

[62] T. Unterthiner, S. Van Steenkiste, K. Kurach, R. Marinier, M. Michalski, and S. Gelly. Towards accurate generative models of video: A new metric & challenges. *arXiv preprint arXiv:1812.01717*, 2018.

[63] J. Hessel, A. Holtzman, M. Forbes, R. L. Bras, and Y. Choi. Clipscore: A reference-free evaluation metric for image captioning. *arXiv preprint arXiv:2104.08718*, 2021.

[64] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017.

Appendix

7 Use of LLMs

LLMs assisted with language editing and with integrating author-provided reviewer responses into the manuscript. The pipeline also uses GPT-5 to compose simulation action primitives, as described in the main text. The authors are responsible for verifying the scientific content, experimental results, and references.

8 Task Details

To facilitate assets alignment between the real-world and simulated settings, we select five representative tasks in RoboTwin [38]—*Shake Bottle*, *Move Playing Card Away*, *Stack Blocks Two*, *Place Mouse Pad* and *Handover Bottle*—to evaluate our compositional world simulation framework. Their respective success criteria are defined as follows.

- *Stack Blocks Two* utilizes two colored blocks—green and yellow. This task is designed primarily to assess the model's ability to generalize to novel spatial configurations. Success is achieved when the robot first places the green block at the designated position and subsequently stacks the yellow block precisely on top of it.
- *Shake Bottle* involves four beverages—Fanta, Coca-Cola, Sprite, and Nongfu Spring Oriental Leaf Tea. Among these beverages, Fanta is employed to collect real-world demonstrations, while the remaining three serve as new objects for an ablation study on model generalization. The task is deemed successful if the robot grasps the bottle from the desktop, lifts it to a predefined height, and performs a shaking motion.
- *Move Playing-Card Away* employs two types of playing cards that differ in color—blue and red. Following the same protocol as *Shake Bottle*, the blue playing card is used to construct the real-world training dataset, whereas the red playing card serves as an object absent from real training demonstrations for evaluating model generalization. The task is considered successful once the robot grasps the designated card and transports it completely away from the central region of the desktop.
- *Place Mouse Pad* requires the robot to grasp a mouse and place it precisely onto a designated colored pad. Success is achieved when the mouse is fully positioned on the target pad.
- *Handover Bottle* is specifically designed to evaluate the model's robustness when handling dynamic objects. The task involves two robotic arms working in coordination. It is considered successful when the robot successfully grasps the bottle and hands it over to another robot in a seamless manner.

To systematically assess the model capabilities beyond the standard task settings, we introduce two types of scene variations for a subset of tasks:

- *Cluttered Scene:* For *Move Playing-Card Away*, *Place Mouse Pad*, and *Handover Bottle*, we introduce task-irrelevant distractors (e.g., office supplies, miscellaneous objects) placed randomly on the desktop. This protocol evaluates the model's ability to distinguish the target object from distractors in complex environments.
- *Background Shift:* For *Move Playing-Card Away* and *Place Mouse Pad*, we alter the environmental background (e.g., changing desktop texture and color). This protocol assesses the model's robustness to contextual variations and ensures it does not overfit to specific background features observed during training.

Data accounting. The neural simulator uses a paired corpus of 200 trajectories in total across 10 source tasks; the policy mixtures instead use 200 synthetic demonstrations per task. The eight downstream settings are Shake Bottle, Stack Blocks Two, Move Playing-Card Away (standard,

cluttered, and colored background), Place Mouse Pad (cluttered and colored background), and Handover Bottle (cluttered). Each task and data regime has its own single-task DP. Additional qualitative figures depict Adjust Bottle, Ranking Blocks RGB, and Stack Blocks Three; these are not additional rows in the downstream benchmark.

9 Training Details

9.1 Neural Simulator Training Details

As mentioned in the main text, our Neural Simulator builds upon Stable Diffusion 1.5 [41], a latent diffusion model with a U-Net denoiser. The simulation video conditions cross-attention; a three-layer MLP encodes actions and injects them through ControlNet residual blocks. Visual and action conditions drop independently with probability 0.1. Inference uses only the simulation video and action sequence, with no task label, and composes separately evaluated scores with $(w_v, w_c) = (1.5, 1.0)$. FRESCO [37] is applied after denoising for temporal correspondence. One neural simulator is shared across tasks.

Neural-simulator and policy training use a single NVIDIA H200 GPU. During fine-tuning, the video data is first converted into image sequences at 10 FPS and organized into a training set, with $1/5$ of the frames randomly sampled as validation in the original experiment. This frame split underlies the historical diagnostics in Appendix 19; adjacent frames may cross the split, so these diagnostics do not establish held-out-trajectory generalization. The model is trained for 30 epochs with a batch size of 8 and a gradient accumulation step of 4, taking approximately 10 hours. We employ the `torch.optim.AdamW` optimizer with a learning rate of 5.0×10^{-5} and a linear warm-up ratio of 0.01. For the loss function, the diffusion model's denoising loss is empirically weighted by 1.0, the perceptual loss (measuring feature-level differences between generated and real images) is weighted by 0.2, and the pixel-wise loss (computing the RGB mean squared error between generated and real images) is weighted by 0.1. During inference, we follow the default parameter settings of FRESCO, except that the minimum key-frame sampling interval is set to 3, to ensure smoother video generation.

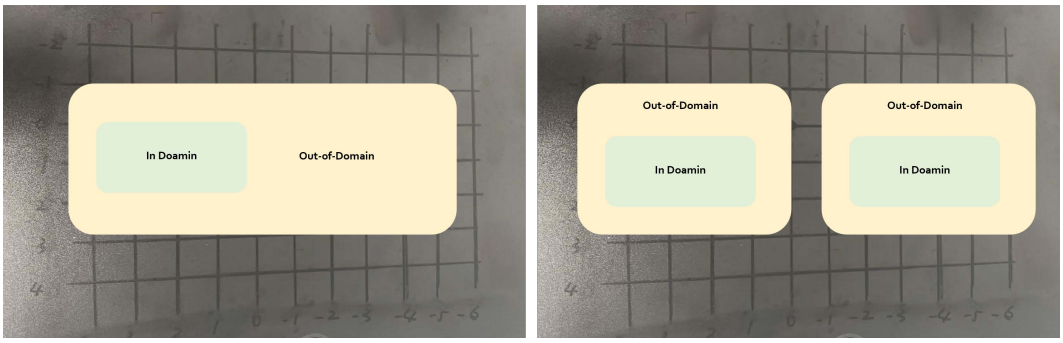
9.2 DP Training Details

9.2.1 Demonstrations

Real-World Demonstrations were meticulously collected via human teleoperation using a pair of PiPER Teach Pendants (see Appendix 10.1). For each task, we recorded only 20 trajectories, all confined to the in-domain spatial and object distribution. Concretely, demonstrations for *Shake Bottle* and *Move Playing-Card Away* were acquired exclusively with the Fanta bottle and the blue playing card starting within the in-domain region illustrated as Fig. 5(a), respectively. And for *Stack Blocks Two*, the green and yellow blocks were always placed in the left and right in-domain zones at the beginning of demonstration collection (Fig. 5(b)). Finally, we randomly selected 10 out of these 20 demonstrations to construct the data-mixture regime *10 Real*, and used all 20 to construct the regime *20 Real*.

Simulated Demonstrations were generated in the traditional simulator RoboTwin, and we collected 200 trajectories for each task. In contrast to the real-world ones, these simulated demonstrations deliberately incorporated out-of-domain spatial arrangements and objects. Specifically, for *Shake Bottle* we used not only the Fanta bottle but also Coca-Cola, Sprite and Nongfu Spring Oriental Leaf Tea, while for *Move Playing-Card Away* we included the red playing card in addition to the blue one. Besides, all objects might be placed in out-of-domain regions when data collection started. All 200 simulated demonstrations were employed to construct the data-mixture regimes *10 Real + 200 Sim* and *200 Sim Pre-train + 10 Real*.

Pseudo-Real Demonstrations were produced by our compositional world-simulation framework under the same out-of-domain spatial and object settings employed for Simulated Demonstrations.



(a) Spatial Distributions of Shake Bottle and Move Playing-Card Away

(b) Spatial Distributions of Stack Blocks Two

Figure 5: Definition of in-domain and out-of-domain spatial distributions in different tasks. Both terms refer exclusively to the initial position of objects before being manipulated. In-domain positions lie within the region covered by real demonstrations; out-of-domain positions lie outside that region. Synthetic policy-training data cover both regions.

The full set of 200 pseudo-real trajectories was used to establish the data-mixture regimes *10 Real + 200 Pseudo-Real* and *200 Pseudo-Real*.

9.2.2 Training Settings

The policy architecture, optimizer, training schedule, and compute budget are reported in Sec. 4.5. The demonstration mixtures and evaluation settings are detailed here for reference.

10 Evaluation Details

10.1 Platform

As depicted in Fig. 6, the real-world evaluations were performed with two ORBBEC PiPER 6-DOF Lightweight Robotic Arms, each equipped with a two-finger gripper (maximum opening 70 mm, gripping force 40 N). A fixed, top-down ORBBEC DaBaiDC1 RGB-D camera provided a global RGB view of the workspace, defined as the central area of a black tabletop. In addition, a pair of PiPER Teach Pendants enabled teleoperation of the arms, allowing efficient collection of real-world demonstrations. All hardware units were connected to a workstation housing an NVIDIA GeForce RTX 4090 GPU, which stored the captured observations, performed model inference, issued control commands, and drove the robotic arms in real time.

10.2 Evaluation Settings

For each ablation dimension—in-domain spatial/object configurations, out-of-domain spatial layouts, and out-of-domain objects—we independently constructed a fixed set of 30 diverse real-world trials. Every policy trained under a different data-mixture regime was evaluated on the corresponding 30-trial split, guaranteeing that all comparisons within a distribution are performed on an identical test bed. Task-success criteria are provided in Appendix 8.

Relationship between neural-simulator training and policy evaluation (data split and leakage). The test bottles (Coca-Cola, Sprite, and Nongfu Spring) and red playing card are absent from the real pairs used to train the neural simulator and from real DP demonstrations. They are present in the digital-twin asset library and in Sim/Pseudo-Real policy-training data. Likewise, spatial-OOD positions are covered by synthetic policy-training data but not by real demonstrations. Real evaluation trajectories are freshly collected, separate from training trajectories. Consequently, the object and spatial experiments evaluate transfer of synthetic coverage to real execution, not extrapolation to

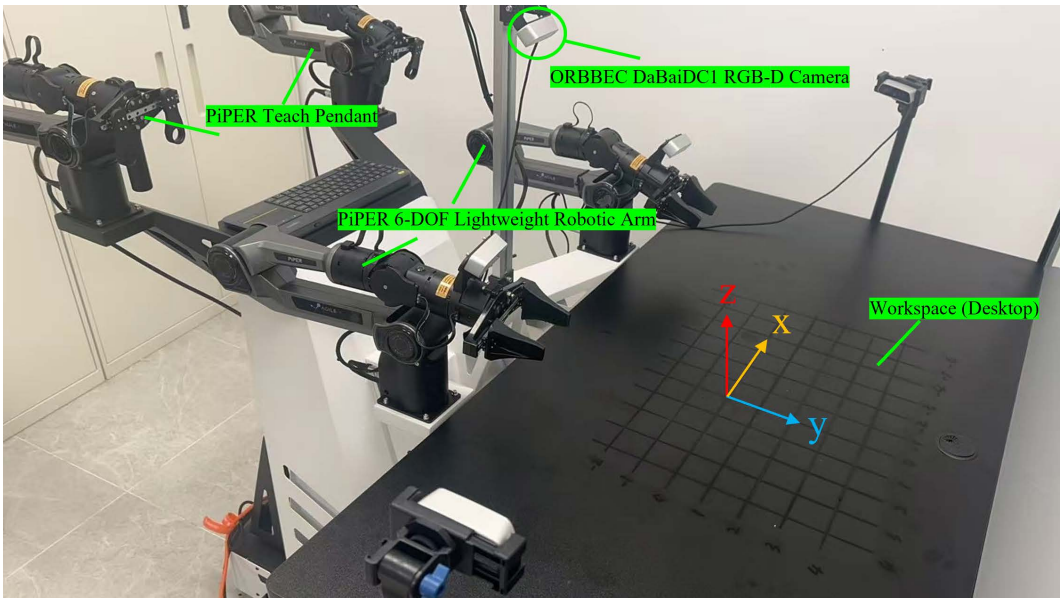


Figure 6: Real-world evaluation platform.

objects or regions unseen by the entire pipeline. Alignment visualizations show calibration examples, not evidence that those real videos were used for neural-simulator training.

11 Sim2Real Neural Simulation Details

As stated in the main text, we enforce strict alignment between the real-world and simulated environments—encompassing background and object appearance, camera intrinsics/extrinsics, and object positions—to enable effective Sim2Real neural simulation. To prevent any learning-induced errors from propagating into the subsequent training of the neural simulator, we adopt a purely rule-based alignment pipeline rather than a data-driven one. Concretely, we first parameterize the relevant attributes of the real-world scene and then transfer the estimated parameters to configure the simulated environment accordingly.

11.1 Background and Object Alignment

Background Alignment mainly parameterizes both the visual appearance of the desktop and the laboratory walls. Using the fixed RGB-D camera described in Sec. 10.1, we first capture images of the table surface and the wall regions. A digital color-picker is then applied to the acquired images to extract representative RGB values.

Regular-Object Alignment covers primitives such as blocks, spheres, and cylinders whose geometry can be described by a small set of metric dimensions. For these instances, we first measure their principal axes (length, width, height, diameter, etc.) with calipers. Their appearance is parameterized by acquiring an orthographic RGB patch of the object's most representative face and extracting the median albedo via a color-picker tool—no additional texture map is required, yielding a compact, error-tolerant representation.

Special-Object Alignment. Owing to RoboTwin's one-to-one digital twins of real-world assets—including Fanta, Coca-Cola, Sprite, Nongfu Spring Oriental Leaf Tea bottles and the playing cards—we can directly pair every physical item with its pre-modeled, dimension- and texture-matched counterpart. This eliminates the need for on-the-fly scanning or manual modeling: each real-world bottle or card is simply mapped to its pre-registered URDF/FBX model, matching nominal geometry

and texture assets, while leaving residual differences in lighting, materials, background, and sensor response.

11.2 Camera Calibration and Alignment

Camera calibration requires intrinsic and extrinsic parameters. The intrinsics can be known directly from its technical documentation; hence, the following section details only the extrinsic-calibration pipeline employed in our setup.

To ensure consistency with RoboTwin, we establish a real-world coordinate system as illustrated in Fig. 6. Within this coordinate system, we first place a calibration checkerboard and obtain the 3D coordinates of its corner points. Subsequently, we capture images using the mounted camera and extract the 2D pixel coordinates of those checkerboard corners via corner detection. The code implementing this procedure is listed below.

```python
import cv2
import numpy as np

# ---------------------------
PATTERN_SIZE = (7, 4)          # Number of checkerboard corners along rows and columns
IMG_PATH     = 'chess.jpg'     # Path of the captured image
# ---------------------------

img  = cv2.imread(IMG_PATH)
if img is None:
    raise FileNotFoundError(IMG_PATH)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 1. Checkerboard Detection
ret, corners = cv2.findChessboardCorners(
    gray, PATTERN_SIZE,
    cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_FAST_CHECK + cv2.CALIB_CB_NORMALIZE_IMAGE
        )
if not ret:
    raise RuntimeError('Checkerboard detection failed!')

corners = cv2.cornerSubPix(
    gray, corners, (11, 11), (-1, -1),
    criteria=(cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001))
pts2d = corners.reshape(-1, 2).tolist()

# 2. Mouse Callback
def on_mouse(event, x, y, flags, param):
    global pts2d, img_show
    if event == cv2.EVENT_LBUTTONDOWN and pts2d:
        idx = min(range(len(pts2d)),
                  key=lambda i: (pts2d[i][0] - x) ** 2 + (pts2d[i][1] - y) ** 2)
        if (pts2d[idx][0] - x) ** 2 + (pts2d[idx][1] - y) ** 2 < 400:
            del pts2d[idx]
            redraw()

def redraw():
    global img_show
    img_show = img.copy()
    for idx, (u, v) in enumerate(pts2d):
        u, v = int(u), int(v)
        cv2.circle(img_show, (u, v), 5, (0, 0, 255), -1)
        cv2.putText(img_show, str(idx), (u + 10, v - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1, cv2.LINE_AA)
    cv2.imshow('interactive', img_show)

# 3. Main Loop
cv2.namedWindow('interactive', cv2.WINDOW_NORMAL)
cv2.setMouseCallback('interactive', on_mouse)
redraw()

print('Usage:')
print('  Left-click on a corner -> delete nearest corner (r restores all)')
print('  Press q -> save current pts2d.txt and exit')
print('  Press r -> restore all originally detected corners')
print('  Close the window (x) to quit without saving')

while True:
    if cv2.getWindowProperty('interactive', cv2.WND_PROP_VISIBLE) < 1:
        break

```

```
    key = cv2.waitKey(30) & 0xFF  # 30 ms timeout to prevent freezing
    if key == ord('q'):
        np.savetxt('pts2d.txt', np.array(pts2d), fmt='%.6f')
        print('Saved pts2d.txt with', len(pts2d), 'points.')
        break
    elif key == ord('r'):
        pts2d = corners.reshape(-1, 2).tolist()
        redraw()

cv2.destroyAllWindows()
```

The saved 2D points must be paired with 3D points in exactly the same order; deleting a detected corner requires removing its corresponding 3D point. The listed intrinsics and camera configuration are setup-specific and must be replaced for a different camera. With the obtained 3D-to-2D correspondences, the camera's extrinsic parameters—position and orientation—can be recovered by solving a Perspective-n-Point (PnP) problem. The implementation is given below.

```
import numpy as np
import cv2

# 1. Given Known Intrinsics
K = np.array([
    [488.8112487792969, 0.0, 317.05938720703125],
    [0.0, 488.8112487792969, 217.4825439453125],
    [0.0, 0.0, 1.0]
], dtype=np.float64)

dist = np.zeros((4, 1))  # Distortion Coefficients; set to zero if no distortion.

# 2. Input
pts3d = np.loadtxt('pts3d.txt')   # N×3, World Coordinates
pts2d = np.loadtxt('pts2d.txt')   # N×2, Pixel Coordinates

assert pts3d.shape[0] == pts2d.shape[0], 'Mismatch in point count!'

# 3. Estimating Extrinsic
ok, rvec, tvec = cv2.solvePnP(
    pts3d.astype(np.float64),
    pts2d.astype(np.float64),
    K, dist, flags=cv2.SOLVEPNP_ITERATIVE
)
if not ok:
    raise RuntimeError('Solve PnP failed! Please verify that the point correspondences
        are correct.')

# Rotation Vector -> Rotation Matrix
R, _ = cv2.Rodrigues(rvec)

R_w2c = R
t_w2c = tvec.ravel()
R_c2w = R_w2c.T
t_c2w = -R_c2w @ t_w2c   # 3×1

position = t_c2w.tolist()

forward = R_c2w[:, 2]
left    = -R_c2w[:, 0]
up      = -R_c2w[:, 1]

print("static_camera_list:")
print("  - name: head_camera")
print("    type: D435")
print(f"    position: [{position[0]:.3f}, {position[1]:.3f}, {position[2]:.3f}]")
print(f"    forward:  [{forward[0]:.3f}, {forward[1]:.3f}, {forward[2]:.3f}]")
print(f"    left:     [{left[0]:.3f}, {left[1]:.3f}, {left[2]:.3f}]")
```

11.3 Object Position Alignment

We emphasize that the real-world frame depicted in Fig. 6 coincides exactly with the world frame employed in RoboTwin. Under this frame, the desktop surface is tessellated into a uniform 5 cm × 5 cm lattice. Object placement is thereby reduced to aligning the object's center of mass with a lattice node; orientation is selected from a prescribed, rule-based catalogue—namely, axis-aligned poses or rotations of 30°, 45°, and 60° about the x- or y-axis. While this discrete parameterization

Table 6: **Conceptual positioning** of *Compositional Simulation* against representative data sources. "Action preserved" means the conditioning action trajectory is retained by construction rather than re-learned. This is a design-level comparison, not a quantitative benchmark.

Approach	No paired sim/real	Reduces appear. gap	Action preserved	Physics source
Raw simulation (RoboTwin)	✓	✗	✓	simulator
Domain randomization	✓	partial	✓	simulator
Unpaired UDA / style transfer	✓	✓	not guaranteed	none
Action-conditioned world models	✓	✓	learned	learned
Real-to-sim-to-real re-render	✗	✓	✓	simulator
***Compositional Simulation* (ours)**	✗	✓	✓	simulator

is admittedly naive, it routinely delivers positional errors below one centimetre and angular errors below one degree. A data-driven, continuous 6-DoF alignment module will be investigated in future work to supersede this manual gridding scheme.

12 Conceptual Positioning

Training, generation, and scene-setup costs are reported in Sec. 4.5. This section compares the pipeline with other data sources at the design level.

Positioning Against Prior Data Pipelines. Tab. 6 situates *Compositional Simulation* conceptually relative to representative alternatives along the axes most relevant to our claims. Full reproductions of external systems and specialized action-conditioned world models remain future work.

13 Extended Related Work

Robotic Simulation. General-purpose simulators such as Isaac Gym [13] and MuJoCo [12] provide physics simulation for robot learning; Isaac Gym supports GPU-parallel training. Frameworks including RoboCasa [9], Habitat [42], AI2-THOR [43], OmniGibson [44], and RoboFactory [11] offer logically structured APIs that standardize action interfaces, enable systematic domain randomization, and facilitate large-scale dataset generation. ManiSkill3 builds upon the open-source SAPIEN simulator [39] for GPU-parallel simulation, emphasizing clean action semantics and deterministic stepping. Despite these advancements, current simulators still fall short of real-world fidelity, with persistent gaps in appearance statistics, sensor characteristics, and contact physics that hinder robust policy transfer to real-world dynamics.

Robot Learning in Manipulation. Specialized policy architectures [40, 45, 46, 47, 48, 49, 50, 51] excel on narrowly defined tasks yet struggle to carry over to new robot embodiments. Generalist policies trained on large, multi-robot corpora aim to improve transfer: RT-1 [52] unifies vision, language, and action in a single transformer; RT-2 [53] jointly finetunes large vision–language models on web and robot data; diffusion-based RDT-1B [54] and π_0 [55] support diverse manipulation tasks. Vision–language–action systems such as OpenVLA [56], CogACT [57], Octo [58], LAPA [59], and OpenVLA-OFT [60] further demonstrate efficient finetuning across robots and sensing modalities. Collectively, these results highlight a data-driven bottleneck: robust cross-task and cross-embodiment generalization hinges on large, diverse, and high-fidelity datasets that faithfully capture real-world appearance, sensing, and physics.

14 Limitations and Future Work

(i) *Appearance, not physics.* The method retains the simulator's action labels and dynamics; it does not close the contact, friction, or compliance gap. Generated pixels can still contain action or identity errors, motivating episode-level quality control. (ii) *Alignment cost.* Each new scene

needs an aligned digital twin, including camera calibration and object-position alignment. (iii) *Compositional guidance.* Visual conditioning drives appearance transfer, while the downstream VD/Full ablation supports a substantial contribution from action conditioning. Guidance offers a tunable action/appearance tradeoff, without guaranteeing physically correct generated videos. (iv) *Evaluation scope.* We evaluate eight tabletop task/scene settings on one platform with single-task DP. The original full task matrix contains single-run counts; additional three-seed studies cover a narrower set of comparisons. Pretrained VLAs, mobile and deformable manipulation, larger paired-data budgets, and force feedback remain untested. A VLA could consume the generated trajectories, but visual pretraining may change the value of re-rendering; we do not claim a measured VLA benefit. (v) *Synthetic coverage.* Object and spatial transfer rely on configurations covered by synthetic training data.

15 Per-Task Qualitative Sim2Real Comparison: Additional Notes

The qualitative comparison in Fig. 4 (main text) is conducted across four tasks chosen to cover variations along two axes: object attributes (category and color) and desktop background. For each method we select frames that either fully present the complete robotic gripper or capture the key interaction moments of each task, using matched frame indices across methods within each task. The simulated objects exhibit visual discrepancies from their real-world counterparts—in particular, the robotic gripper appears dark black in reality, as opposed to white-gray in the simulator. Conventional diffusion baselines exhibit reasonable scene understanding capability but fail to preserve critical visual semantic information during generation, resulting in severe hallucination artifacts. Ours-CD accurately reproduces the target manipulation actions, yet lacks the scene semantic constraints provided by visual dynamics, so the viewpoint, object instances, and desktop background are generated from the model's inherent prior knowledge. Ours-VD contributes the majority of the sim-to-real transfer capability, while still exhibiting non-negligible flaws in action timing and physical consistency (e.g. motion blur matching arm movement). Ours-Full combines photorealistic visual generation with motion guidance from control dynamics, realizing precise sim-to-real alignment across both perception and action.

16 Pipeline Pseudo-code

Alg. 1 summarizes the end-to-end deployment loop sketched in Sec. 4.2 and Fig. 3: the trained neural simulator $\mathcal{N}$ rewrites each classical-simulator pair $(\mathcal{V}_{\text{sim}}, \mathcal{A})$ into a Pseudo-Real pair, which is then mixed with a small set of real demonstrations under a ratio α to train the downstream Diffusion Policy.

Algorithm 1 Policy Training with Compositional Simulation

Require: Simulated dataset $D_{\text{sim}} = \{(\mathcal{V}_i, \mathcal{A}_i)\}_{i=1}^{N}$
Require: Real demonstration dataset D_{real}, trained translator $f_{\mathcal{N}}$
Require: Real-data sampling probability $\alpha \in [0, 1]$
$D_{\text{pseudo}} \leftarrow \emptyset$
for each $(\mathcal{V}_i, \mathcal{A}_i) \in D_{\text{sim}}$ **do**
    $\widetilde{\mathcal{V}}_i \leftarrow f_{\mathcal{N}}(\mathcal{V}_i, \mathcal{A}_i)$
    $D_{\text{pseudo}} \leftarrow D_{\text{pseudo}} \cup \{(\widetilde{\mathcal{V}}_i, \mathcal{A}_i)\}$
end for
Train DP by sampling real demonstrations with probability α
    and Pseudo-Real demonstrations with probability $1 - \alpha$
return Trained policy π_{robot}

17 Generalization: Rollout Visualizations

This section provides rollout visualizations supporting the quantitative results reported in Sec. 4.3 and Tab. 3. Fig. 7 illustrates spatial generalization on *Move Playing-Card Away*, and Fig. 8 shows new-object generalization on *Shake Bottle*.



Figure 7: Visualization of DP performance on *Move Playing-Card Away*. Top two rows: in-domain spatial distribution. Middle two rows: out-of-domain spatial distribution. Bottom four rows: colored background with varying levels of object clutter. Policies are trained under *20 Real* and *10 Real + 200 Pseudo Real*, respectively.

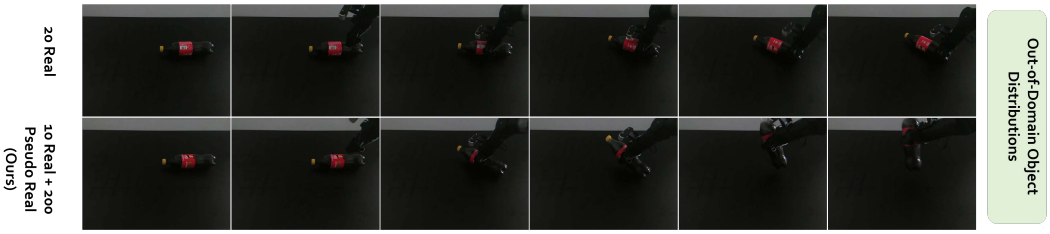


Figure 8: Generalization visualization of DP on *Shake Bottle* under out-of-domain object distributions. Top: policy trained with *20 Real*. Bottom: policy trained with *10 Real + 200 Pseudo Real*.

18 More Result Visualization

18.1 Visualization of Generalization on New Objects

To highlight the enhanced generalization of DP enabled by our compositional world simulation pipeline, we provide trajectory visualizations of the *Move Playing-Card Away* task with additional examples in Fig. 9.

18.2 Visualization of Real2Sim Alignment

As detailed in Appendix 11, we performed exhaustive Real2Sim alignment. Here we illustrate the final alignment quality for the tasks *Shake Bottle*, *Ranking Blocks RGB*, *Stack Blocks Three* and *Stack Blocks Two* in Fig. 10 and Fig. 11.

18.3 Visualization of Sim2Real Neural Simulation

To dynamically demonstrate the effectiveness of our approach in sim-to-real transfer, we further present a visual comparison between the pseudo-realistic videos generated by our Neural Simulator and the initial simulation videos, as shown in Fig. 12. The figure includes *Move Playing-Card Away* and the additional qualitative tasks *Adjust Bottle*, *Stack Blocks Three*, and *Ranking Blocks RGB*. The results indicate that our method consistently maintains strong temporal coherence and perceptual realism throughout the video sequences.



Figure 9: Generalization visualization of DP on new objects. The top two rows correspond to *Move Playing-Card Away*, and the bottom two rows correspond to *Shake Bottle*, respectively.



Figure 10: Real2Sim bottle alignment examples (not neural-simulator training footage). From top to bottom: Nongfu Spring Oriental Leaf Tea, Coca-Cola, Sprite, and Fanta.



Figure 11: Real2Sim alignment on additional tasks. From top to bottom: *Ranking Blocks RGB*, *Stack Blocks Three* and *Stack Blocks Two*.



Figure 12: Sim2Real visualization on various tasks. From top to bottom: *Adjust Bottle*, *Move Playing-Card Away*, *Stack Blocks Three* and *Ranking Blocks RGB*.

19 Historical Frame-Split Diagnostics

The following initial-submission metrics used a random frame split, rather than a trajectory-disjoint split. They are retained for transparency only. LPIPS [61] measures perceptual distance and FVD [62] measures video-distribution distance. CLIPScore [63] measures image–text compatibility, while FID [64] measures image-distribution distance. Baseline denotes SD1.5 video-to-video translation with FRESCO; Zero-Shot denotes the backbone without sim-to-real fine-tuning; Ours-CD and Ours-VD use control and visual conditioning alone, respectively; Ours-Full composes both scores.

Table 7: Historical frame-split visual metrics from the initial submission. Adjacent frames may cross the split; these values are not evidence of held-out-trajectory generalization.

	PSNR ↑	SSIM ↑	CLIP Score ↑	LPIPS ↓	FID ↓	FVD ↓
Sim	16.443	0.4342	0.7564	0.3629	187.40	61.048
Baseline	16.849	0.5129	0.7526	0.3494	254.59	50.369
Zero-Shot	13.093	0.5487	0.7308	0.4756	219.74	163.83
Ours-CD	8.4640	0.1486	0.7216	0.8130	434.44	239.13
Ours-VD	18.153	0.5916	0.7884	0.2813	153.12	22.311
Ours-Full	**19.577**	**0.6484**	**0.8102**	**0.2647**	**147.90**	**15.765**